Assignment 11:

Name: Leon Lobo
Admission No: U24AI043
Github: https://github.com/LeonSimonLobo/AI-lab

1. Consider the following districts in the map of Gujarat.Color this map with the lowest number of colors posing it as a constraint satisfaction problem so that no two adjacent districts are colored the same. [Diu is not a district of Gujarat and Rann of Kuchchh and Little Rann of Kuchchh are also not districts]

Code:

```
import random


def generate_letters(k):
```

```
    letters=list("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
    random.shuffle(letters)
    return letters[:k]

nodes=list({
    "Kachchh","Banaskantha","Patan","Mehsana","Sabarkantha","Gandhinagar",
    "Ahmedabad","Kheda","Panchmahal","Dahod","Vadodara","Anand","Bharuch",
    "Narmada","Surat","Navsari","Valsad","Dang","Surendranagar","Rajkot",
    "Jamnagar","Amreli","Bhavnagar","Junaghad","Porbandar"
})

index={nodes[i]: i for i in range(len(nodes))}
n=len(nodes)

matrix=[[0]*n for _ in range(n)]

edges=[
    ("Kachchh","Banaskantha"),("Kachchh","Patan"),("Kachchh","Surendranagar"),
    ("Banaskantha","Patan"),("Banaskantha","Sabarkantha"),
    ("Patan","Mehsana"),("Patan","Surendranagar"),
    ("Mehsana","Sabarkantha"),("Mehsana","Gandhinagar"),("Mehsana","Ahmedabad"),
    ("Sabarkantha","Gandhinagar"),("Sabarkantha","Panchmahal"),
    ("Gandhinagar","Ahmedabad"),("Gandhinagar","Kheda"),
    ("Ahmedabad","Kheda"),("Ahmedabad","Anand"),("Ahmedabad","Surendranagar"),
    ("Kheda","Anand"),("Kheda","Panchmahal"),
    ("Panchmahal","Dahod"),("Panchmahal","Vadodara"),
    ("Vadodara","Anand"),("Vadodara","Bharuch"),("Vadodara","Narmada"),
    ("Anand","Bharuch"),
    ("Bharuch","Narmada"),("Bharuch","Surat"),
    ("Narmada","Surat"),
    ("Surat","Navsari"),("Surat","Dang"),
    ("Navsari","Valsad"),("Navsari","Dang"),
    ("Surendranagar","Rajkot"),

("Rajkot","Jamnagar"),("Rajkot","Amreli"),("Rajkot","Bhavnagar"),("Rajkot","Porbandar"),("
Rajkot","Junaghad"),
    ("Amreli","Bhavnagar"),("Amreli","Junaghad"),
    ("Porbandar","Junaghad"),
    ("Bhavnagar","Ahmedabad")
]

for u,v in edges:
    i,j=index[u], index[v]
    matrix[i][j]=1
    matrix[j][i]=1
```

```python
def is_valid(node, color, assignment):
    i=index[node]
    for j in range(n):
        if matrix[i][j]==1:
            neighbor=nodes[j]
            if neighbor in assignment and assignment[neighbor]==color:
                return False
    return True

def backtrack(assignment,colors):
    result=False
    if len(assignment)==n:
        return assignment
    for node in nodes:
        if node not in assignment:
            break
    for color in colors:
        if is_valid(node, color, assignment):
            assignment[node]=color
            result=backtrack(assignment,colors)
            if result:
                return result
            del assignment[node]
    return None

def main():
    solution=False
    num_color=1
    while not solution:
        colors=generate_letters(num_color)
        solution=backtrack({},colors)
        num_color+=1

    print("Solution Found:\n")
    for district in solution:
        print(f"{district} → {solution[district]}")
    print("Number of colours used:",num_color-1)

if __name__=='__main__':
    main()
```

Output:

```
Solution Found:

Ahmedabad → B
Gandhinagar → G
Kachchh → B
Surendranagar → V
Valsad → B
Sabarkantha → B
Jamnagar → B
Bharuch → B
Rajkot → G
Junaghad → V
Mehsana → V
Dang → B
Bhavnagar → V
Surat → V
Narmada → G
Porbandar → B
Anand → G
Panchmahal → G
Patan → G
Vadodara → V
Navsari → G
Amreli → B
Kheda → V
Banaskantha → V
Dahod → B
Number of colours used: 3
```

2. Solve the following cryptarithmetic problem posing it as a constraint satisfaction problem. Each letter represents a unique digit from 0 to 9. The leading letters (S and M) cannot be 0. Solve:

```
  SEND
+ MORE
-------
 MONEY
```

Code:

```
letters=["D","E","Y","N","R","O","S","M"]

def is_valid_partial(assign):
    if 'S' in assign and assign['S']==0:
        return False
    if 'M' in assign and assign['M']==0:
        return False

    if all(k in assign for k in ['D','E','Y']):
        if (assign['D'] + assign['E']) % 10 !=assign['Y']:
            return False

    if all(k in assign for k in ['N','R','E']):
        c1=(assign['D'] + assign['E']) // 10 if all(k in assign for k in ['D','E']) else 0
        if (assign['N'] + assign['R'] + c1) % 10 !=assign['E']:
            return False

    if all(k in assign for k in ['E','O','N']):
        c1=(assign['D'] + assign['E']) // 10 if all(k in assign for k in ['D','E']) else 0
        c2=(assign['N'] + assign['R'] + c1) // 10 if all(k in assign for k in ['N','R'])
else 0
        if (assign['E'] + assign['O'] + c2) % 10 !=assign['N']:
            return False

    if all(k in assign for k in ['S','M','O']):
        c1=(assign['D'] + assign['E']) // 10 if all(k in assign for k in ['D','E']) else 0
        c2=(assign['N'] + assign['R'] + c1) // 10 if all(k in assign for k in ['N','R'])
else 0
        c3=(assign['E'] + assign['O'] + c2) // 10 if all(k in assign for k in ['E','O'])
else 0
        if (assign['S'] + assign['M'] + c3) % 10 !=assign['O'] or (assign['S'] +
assign['M'] + c3) / 10 !=1:
            return False

    if 'M' in assign:
        if assign['M'] !=1:
            return False

    return True


def backtrack(assign, used):
    if len(assign)==8:
        return assign
    for var in letters:
```

```
        if var not in assign:
            break
    for d in range(10):
        if d not in used:
            assign[var]=d
            used.add(d)

            if is_valid_partial(assign):
                result=backtrack(assign, used)
                if result:
                    return result

            del assign[var]
            used.remove(d)

    return None


def main():
    assign={'M':1}
    solution=backtrack(assign, set([1]))

    print("Solution:", solution)

    D,E,Y,N,R,O,S,M=[solution[l] for l in letters]

    SEND=1000*S + 100*E + 10*N + D
    MORE=1000*M + 100*O + 10*R + E
    MONEY=10000*M + 1000*O + 100*N + 10*E + Y

    print("\n", SEND)
    print("+", MORE)
    print("------")
    print(MONEY)


if __name__=='__main__':
    main()
```

Output:

```
Solution: {'M': 1, 'D': 7, 'E': 5, 'Y': 2, 'N': 6, 'R': 8, 'O': 0, 'S': 9}

 9567
+ 1085
------
10652
```